testing | Status: mixed | App v1.0.20 | Reviewed 2026-10-24

Testing

Part A — Current state [current]

A.1 Automated tests

Check	Status
Unit tests (`app/src/test`)	**None.** The source set doesn't exist.
Instrumented tests (`app/src/androidTest`)	**None.**
Test dependencies	Declared: JUnit 4.13.2, AndroidX JUnit 1.2.1, Espresso 3.6.1, Compose `ui-test-junit4` , `hilt-android-testing` , `ui-test-manifest` (debug)
Instrumentation runner	`androidx.test.runner.AndroidJUnitRunner` (configured, nothing uses it)
CI	`.github/workflows/build_android.yaml` runs **only** `./gradlew assembleRelease` . No tests, no lint.
Static analysis / formatter	No ktlint, detekt, ktfmt, or Android Lint config in the repo

A.2 Test seams in production code

The code has explicit test hooks. Their comments say "Overridden in tests", which suggests a test suite exists upstream but isn't published here **[inferred]**:

Hook	Location	Purpose
`localTestAllowlistFile` (`@VisibleForTesting`)	`ModelManagerViewModel`	Point allowlist loading at a fixture
`allowlistLoadingDispatcher` (`@VisibleForTesting`)	`ModelManagerViewModel`	Load the allowlist synchronously in tests
Secondary constructor without `downloaders`	`ModelManagerViewModel`	Construct without Hilt multibindings
`testingModelHelper`	`runtime/ModelHelperExt.kt`	Swap in a fake `LlmModelHelper` for every model
`ioDispatcher` (internal var)	`LlmChatModelHelper`	Control I/O in tests
`setModelLimitsForTesting`	`UniversalOnDeviceEmbedder`	Bypass reading model metadata
`open` classes (`ModelManagerViewModel` , `DefaultAgentRuntimeExecutor` , `HuggingFaceApiClient` , `RuntimeToolsProvider` , ...)	various	Subclass-based fakes

A.3 Manual and developer testing aids

Aid	How
Local model catalog	`adb push dev.json /data/local/tmp/model_allowlist_test.json`. It overrides the network allowlist, and the app shows a warning. Delete the file to revert.
Side-loaded models	Push to `/data/local/tmp/...`, then import in the Model manager, or use `localModelFilePathOverride`
Test chat task	`llm_test` runs any imported model in full chat
Skill tester	Agent Skills, then Skill manager, then test a skill's scripts in-app (`SkillTesterBottomSheet`)
JS skill debugging	WebView console output goes to logcat (`AGGalleryWebView`). See skills/README.md.
Benchmark	Built-in benchmark screen for LiteRT-LM models
Logs	Tags are prefixed `AG…`. Filter with `adb logcat
Bug reports	Bug_Reporting_Guide.md

Part B — Proposed strategy [proposed]

Nothing in Part B exists yet. Priorities are ordered by risk reduction per unit of effort.

B.1 Test pyramid

Layer	Scope	Tooling	Runs in
Unit (JVM)	Pure logic: parsing, mapping, thresholds, URL building	JUnit 4, `kotlinx-coroutines-test`, Truth or plain asserts	CI on every PR
ViewModel	State transitions with fake repositories and `testingModelHelper`	JUnit + `StandardTestDispatcher`	CI
Instrumented	DataStore round-trips, WebView JS skill contract, Compose navigation	AndroidX Test, Compose UI test, Hilt testing	CI emulator (nightly)
Device / model	Real model init and inference per accelerator (CPU, GPU, NPU)	Benchmark screen, manual matrix	Pre-release

B.2 First tests to write (highest value)

#	Target	Why	Notes
1	`ModelManagerViewModel.getTokenStatusAndData`	Would catch the 5-minute buffer bug (subtracts 300 ms)	See 14-known-issues

#	Target	Why	Notes
2	`AllowedModel.toModel()`	Catalog drives every feature: accelerator parsing, Pixel NPU→TPU, Pixel 10 GPU removal, AICore temperature cap, per-SoC files, default URL	Pixel checks need injectable device info
3	`loadModelAllowlist()` source order and filtering	Test-file override, network, then cache; disabled, AICore, and NPU filtering	Use `localTestAllowlistFile` and `allowlistLoadingDispatcher`
4	`SkillManager.convertSkillMdToProto`	Untrusted input (remote and local skills)	Missing name/description, `---` inside body, metadata keys
5	`Skill.getJsSkillUrl` / `getJsSkillWebviewUrl`	Origin selection (local appassets vs remote) is security-relevant	
6	`RunJsTool` result parsing	`result` / `error` / `image` / `webview` / unparsable branches	Fake `SkillsProvider` and action channel
7	`IntentHandler` parameter parsing	Model-produced JSON is often malformed	Calendar date formats

#	Target	Why	Notes
8	`SummarizationContextCompactor.isOverTokenThreshold`	Simple threshold maths; guards future enablement	
9	`DownloadWorker` Content-Range parsing and zip-slip guard	Extract to pure functions first	Security-relevant
10	Allowlist JSON validation	Catch a bad catalog before it ships to every user	Small script or JVM test that parses every `model_allowlists/*.json` into `ModelAllowlist`

Example (target 4):

```
        // app/src/test/java/com/google/ai/edge/gallery/skills/SkillMdParserTest.kt
        class SkillMdParserTest {
          @Test fun parsesNameDescriptionAndSecret() {
            val md = "---\nname: hash\ndescription: Hash text\nmetadata:\n  require-secret:
true\n---\nCall run_js."
            val (skill, errors) = SkillManager.convertSkillMdToProto(md, builtIn = false,
selected = true)
            assertTrue(errors.isEmpty())
            assertEquals("hash", skill!!.name)
            assertTrue(skill.requireSecret)
            assertEquals("Call run_js.", skill.instructions)
          }

          @Test fun rejectsMissingDescription() {
            val (skill, errors) = SkillManager.convertSkillMdToProto("---\nname: x\n---
\nbody", false, true)
            assertNull(skill)
            assertTrue(errors.any { it.contains("description") })
          }
        }
```

B.3 CI changes

Add to `build_android.yaml`, after `assembleRelease`:

```
      - name: Unit tests
        run: ./gradlew testDebugUnitTest
      - name: Lint
        run: ./gradlew lintDebug
```

Also add a job that validates `model_allowlists/*.json` and `skills/**/SKILL.md` on PRs touching those paths. The allowlist ships to users without an app release, so it needs its own gate.

B.4 Release test checklist (manual, per release)

- ❑ Fresh install → home loads the allowlist for the new `versionName` (no "Failed to load model list").
- ❑ Download, init, chat for one model on CPU and GPU, and on NPU where supported.
- ❑ Gated-model download through Hugging Face OAuth.
- ❑ Ask Image (multi-image), Audio Scribe (30 s clip).
- ❑ Agent Skills: a built-in JS skill (e.g. calculate-hash), an intent skill (create-calendar-event), skill with a secret, MCP server with a header.
- ❑ Instant Media Search: sample album index and search. Video Moment Finder: process, search, trim.
- ❑ Interrupted download resumes after relaunch.
- ❑ Deep link `com.google.ai.edge.gallery://llm_agent_chat?query=hi` shows the confirmation dialog before sending.
- ❑ Analytics opt-out persists.